



ESOPTRON

Badge Sceau de Salomon x Cube de Métatron

Spécification technique & méthode d'intégration complète

Projet	Valeur
Dépôt	github.com/oykdo/esoptron
Module cible	src/eopx/metatron/seal_overlay.py
Format sortie	.eopx (PNG + chunks ML-DSA-87)
Stack	Python 3.11+ / Pillow / pqcrypto / ArUco
Auteur	Eidolon Team
Version	1.0.0 — Mai 2026

1. Contexte et objectif

Esoptron encode déjà les identités de coffre-fort cryptographiques dans la géométrie du Cube de Métatron ($F_{13} = 91$ symboles). Le présent handover décrit comment superposer le Sceau de Salomon (hexagramme à 6 branches) sur ce même template pour générer un badge visuel enrichi, à la fois lisible par un humain et vérifiable par machine, exporté en fichier .eopx signé ML-DSA-87.

Pertinence géométrique : Le Sceau de Salomon est géométriquement inscrit dans le Cube de Métatron — les deux triangles entrelacés émergent naturellement de l'intersection des 13 cercles. La superposition n'est donc pas un ajout cosmétique mais une révélation structurelle de la géométrie sous-jacente.

1.1 Correspondances géométriques

Zone géométrique	Donnée encodée	Protocole Esoptron
6 pointes du Sceau	6 segments du vault_id	Protocol A — Unlock
Hexagone central	spinor_hash (phase 6, 64B)	Protocol B — Verify
Triangle ascendant	ML-DSA-87 pk fingerprint	Signature post-quantique
Triangle descendant	ML-KEM-1024 pk fingerprint	KEM post-quantique
Cercles Fruit of Life	ArUco markers (ID 0-12)	Détection machine (CV)
Périmètre externe	merkle_root + timestamp	Protocol F — Migrate

2. Architecture du module

2.1 Arborescence cible

Le module s'intègre dans la hiérarchie existante sans modifier les fichiers existants :

```
src/eopx/metatron/
  __init__.py      # existant
  encode.py        # existant — F13 91 symboles
  aruco.py         # existant — markers CV
  seal_overlay.py  # NOUVEAU — ce module
  geometry.py      # NOUVEAU — primitives géométriques
  color_map.py     # NOUVEAU — dérivation HSL depuis hash

scripts/
  eopx_badge.py    # NOUVEAU — CLI dédié badge

tests/
  test_seal_overlay.py # NOUVEAU — suite de tests
```

2.2 Flux de traitement complet

Le pipeline d'un badge complet se déroule en 5 étapes séquentielles :

N°	Étape	Description
1	Extraction des données vault	Lecture vault_id, spinor_hash, pk_dsa, pk_kem, merkle_root depuis Eidolon
2	Calcul de la géométrie	geometry.py — calcul des 91 points F ₁₃ , hexagramme, apex des 6 pointes
3	Dérivation chromatique	color_map.py — HSL depuis spinor_hash : teinte unique par vault_id
4	Rendu SVG + rasterisation	seal_overlay.py — SVG paramétrique, conversion PNG via Pillow/cairosvg
5	Signature .eopx	eopx_pack.py existant — injection chunks + signature ML-DSA-87

3. Spécification technique

3.1 Module geometry.py

Ce module fournit les primitives de calcul géométrique pour le Cube de Métatron et le Sceau de Salomon.

Classe principale : MetatronGeometry(center, radius)

Méthode	Retourne
fruit_of_life_centers()	13 centres (x,y) des cercles — rayon = radius/2 chacun
metatron_lines()	78 lignes reliant chaque centre à tous les autres
hexagram_points()	12 points (6 apex + 6 jointures) du Sceau de Salomon
triangle_up()	3 apex du triangle ascendant (feu)
triangle_down()	3 apex du triangle descendant (eau)
inner_hexagon()	6 sommets de l'hexagone central
aruco_positions()	13 positions {id, x, y, size} pour placement ArUco

3.2 Module color_map.py

Dérive une palette chromatique cohérente et reproductible depuis le spinor_hash du vault.

Algorithme de dérivation :

- Extraire les 3 premiers octets du spinor_hash (bytes 0-2)
- Octets 0-1 → teinte HSL : $H = (b0 \ll 8 \mid b1) \% 360$
- Octet 2 → saturation : $S = 50 + (b2 \% 30) \rightarrow [50\%, 80\%]$
- Luminosité fixe : $L = 35\%$ (triangle ascendant) / 55% (descendant) / 75% (hexagone)
- Couleur ArUco : dérivée de bytes 3-5, toujours sombre ($L < 30\%$) pour contraste CV

La palette est déterministe : deux coffres avec le même spinor_hash produiront exactement les mêmes couleurs. La variation chromatique est donc une authentification visuelle de premier niveau.

3.3 Module seal_overlay.py — API principale

Fonction principale : render_seal_badge(vault_data: dict, size: int = 1024) -> Image

Paramètre vault_data (dict) :

Clé	Type	Source
vault_id	str (UUID)	Eidolon Phase 1
spinor_hash	bytes (64)	Eidolon Phase 6
merkle_root	bytes (32)	Eidolon Phase 9
pk_dsa	bytes (ML-DSA-87)	eopx_keygen.py

pk_kem	bytes (ML-KEM-1024)	eopx_keygen.py
--------	---------------------	----------------

3.4 Couches de rendu SVG

Le SVG est construit en 7 couches superposées (ordre de rendu) :

Z	Couche	Opacité	Contenu
0	Fond	100%	Cercle plein — couleur dérivée L=95%
1	Fruit of Life	30%	13 cercles — trait fin, couleur palette
2	Lignes Métatron	15%	78 lignes — strokeWidth=0.5px
3	Triangle descendant	60%	Fill couleur eau, stroke épais
4	Triangle ascendant	60%	Fill couleur feu, stroke épais
5	Hexagone central	80%	Fill solide, contient hash tronqué
6	ArUco markers	100%	13 markers aux centres F_{13} , détectables CV

4. Guide d'implémentation

4.1 Dépendances à ajouter

Ajouter dans pyproject.toml, section [project.dependencies] :

```
cairosvg = ">=2.7"          # SVG → PNG haute qualité
svgwrite = ">=1.4"          # Construction SVG programmatique
```

cairosvg requiert libcairo sur le système. Sur Debian/Ubuntu : apt install libcairo2. Sur macOS : brew install cairo. Sur Alpine (iSH) : apk add cairo.

4.2 Squelette de seal_overlay.py

Structure minimale du module principal :

```
from __future__ import annotations
import hashlib, math
from PIL import Image
import svgwrite, cairosvg
from .geometry import MetatronGeometry
from .color_map import derive_palette
from .aruco import place_aruco_markers

def render_seal_badge(
    vault_data: dict,
    size: int = 1024,
    show_hash_label: bool = True,
) -> Image.Image:
    geo = MetatronGeometry(center=(size//2, size//2), radius=size*0.42)
    palette = derive_palette(vault_data['spinor_hash'])
    dwg = svgwrite.Drawing(size=(size, size))
    _draw_background(dwg, geo, palette)
    _draw_fruit_of_life(dwg, geo, palette)
    _draw_metatron_lines(dwg, geo, palette)
    _draw_triangles(dwg, geo, palette, vault_data)
    _draw_hexagon(dwg, geo, palette, vault_data)
    place_aruco_markers(dwg, geo, vault_data)
    png = cairosvg.svg2png(bytestring=dwg.tostring(), output_width=size)
    return Image.open(io.BytesIO(png))
```

4.3 CLI eopx_badge.py

Script en ligne de commande intégrant le badge dans le pipeline .eopx existant :

```
python scripts/eopx_badge.py \
    --vault-id <UUID> \
    --key keys/default.json \
    --spinor-hash <hex64> \
    --merkle-root <hex32> \
```

```
--size 1024 \  
--out badge_vault.eopx
```

Le script appelle successivement :

- `render_seal_badge()` → PNG en mémoire
- `eopx_pack.pack(image=png, key=key_json)` → .eopx signé
- `eopx_verify.verify(badge_vault.eopx)` → validation intégrité

4.4 Vérification du badge

Le badge généré est un .eopx standard, vérifiable avec l'outil existant :

```
python scripts/eopx_verify.py badge_vault.eopx
```

Et via le SDK TypeScript sans installation Python :

```
import { verifyChunksOnly } from '@esoptron/verify';  
const result = verifyChunksOnly(fs.readFileSync('badge_vault.eopx'));  
console.log(result.ok, result.manifest.vaultId);
```

5. Stratégie de tests

5.1 Couverture minimale requise

Test	Criticité	Critère de succès
Déterminisme du rendu	HAUTE	2 appels même input → PNG identiques (SHA-256)
Détection ArUco 13/13	HAUTE	cv2.aruco.detectMarkers trouve les 13 IDs
Signature .eopx valide	HAUTE	eopx_verify.py retourne OK
Unicité chromatique	MOYENNE	10 vault_id différents → 10 palettes distinctes
Taille PNG correcte	BASSE	Image.size == (size, size)
Rendu mobile (256px)	BASSE	PNG lisible sans artefact à 256×256

5.2 Commandes de test

```
# Lancer la suite complète  
python -m pytest tests/test_seal_overlay.py -v
```

```
# Test de détection ArUco seule  
python -m pytest tests/test_seal_overlay.py::test_aruco_detection -v
```

```
# Générer un badge de référence visuelle  
python scripts/eopx_badge.py --demo --out demo_badge.eopx
```

6. Checklist de livraison

6.1 Fichiers à créer

- `src/eopx/metatron/geometry.py`
- `src/eopx/metatron/color_map.py`
- `src/eopx/metatron/seal_overlay.py`
- `scripts/eopx_badge.py`
- `tests/test_seal_overlay.py`

6.2 Fichiers à modifier

- `src/eopx/metatron/__init__.py` — exporter `render_seal_badge`
- `pyproject.toml` — ajouter `cairosvg`, `svgwrite`
- `README.md` — section Badge Generation
- `docs/whitepaper_esoptron.md` — appendice géométrie

6.3 Points de vigilance

Le module ne doit PAS modifier le comportement de `eopx_pack.py` existant. L'image PNG générée par `render_seal_badge()` est passée comme argument, sans altération du pipeline de signature.

Les ArUco markers doivent utiliser le dictionnaire `DICT_4X4_50` (déjà utilisé dans `aruco.py` existant) pour maintenir la compatibilité avec le scanner PWA.

Sur les builds CI/CD sans interface graphique, `cairosvg` peut nécessiter la variable d'environnement `DISPLAY=:0` ou l'utilisation de `Xvfb`. Documenter dans `CONTRIBUTING.md`.

7. Intégration avec Eidolon

Le badge Sceau de Salomon s'insère dans le flux Eidolon → Esoptron documenté dans le whitepaper :

Eidolon (source)	Badge (mapping)	Protocole Esoptron
Phase 6 : <code>spinor_hash</code> (64B)	Palette HSL + hexagone central	Protocol B — Verify
Phase 9 : <code>merkle_root</code> (32B)	Périmètre externe du badge	Protocol F — Migrate
<code>machine_lock</code> binding	ArUco ID 0 (centre)	Protocol D — Enroll
ML-DSA-87 pk	Triangle ascendant	Signature chunk eopx
ML-KEM-1024 pk	Triangle descendant	KEM chunk eopx

Le badge produit est un `.eopx` valide : il peut être utilisé directement dans tous les Protocoles A-F sans adaptation, le scanner PWA le traite comme n'importe quel autre fichier `.eopx`.

✦ ἔσοπτρον — la surface qui reflète sans révéler ✦